Algorithm Development and Control Statements

Visual C# How to Program, 6/e

Algorithms

- Computers solve problems by executing a series of actions in a specific order.
- An algorithm is a procedure for solving a problem, in terms of:
  - the actions to be executed and
  - the order in which these actions are executed

Algorithms (Cont..)

- Consider the “rise-and-shine algorithm” followed by one junior executive for getting out of bed and going to work:
  - (1) get out of bed,
  - (2) take off pajamas,
  - (3) take a shower,
  - (4) get dressed,
  - (5) eat breakfast and
  - (6) carpool to work.

Algorithms (Cont.)

- Suppose that the same steps are performed in a different order:
  - (1) get out of bed,
  - (2) take off pajamas,
  - (3) get dressed,
  - (4) take a shower,
  - (5) eat breakfast,
  - (6) carpool to work.
- In this case, the junior executive shows up for work soaking wet.
- The order in which statements execute is called program control.

Pseudocode

- Pseudocode is similar to English—it's not a programming language.
- It helps you "think out" an app before attempting to write it.
- A carefully prepared pseudocode app can easily be converted to a corresponding C# app.

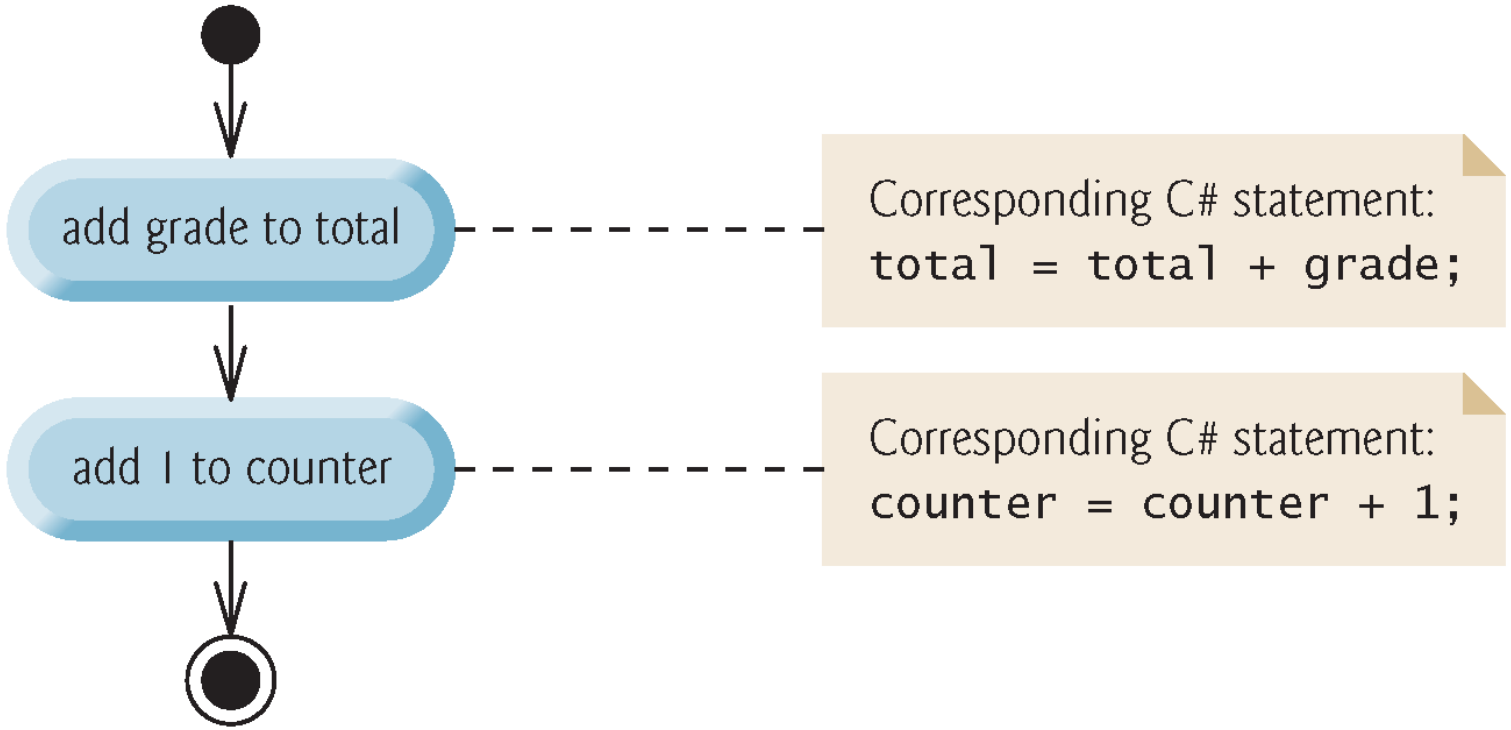
Pseudocode

- The pseudocode of Fig. 5.1 corresponds to the algorithm that inputs two integers from the user, adds these integers and displays their sum.
- The pseudocode statements are simply English statements that convey what task is to be performed in C#.

```
1  Prompt the user to enter the first integer
2  Input the first integer
3
4  Prompt the user to enter the second integer
5  Input the second integer
6
7  Add first integer and second integer, store result
8  Display result
```

Control Structures

- Normally, statements execute one after the other in sequential execution.
- Various C# statements enable you to specify the next statement to execute. This is called transfer of control.
- Structured apps are clearer, easier to debug and modify, and more likely to be bug free.



Selection Statements

- C# has three types of selection structures, which from this point forward we shall refer to as **selection statements**.
  - The **`if` statement** performs (selects) an action if a condition is *true* or skips the action if the condition is *false*.
  - The **`if...else` statement** performs an action if a condition is *true* or performs a different action if the condition is *false*.
  - The `switch` statement performs one of many different actions, depending on the value of an expression.

Iteration Statements

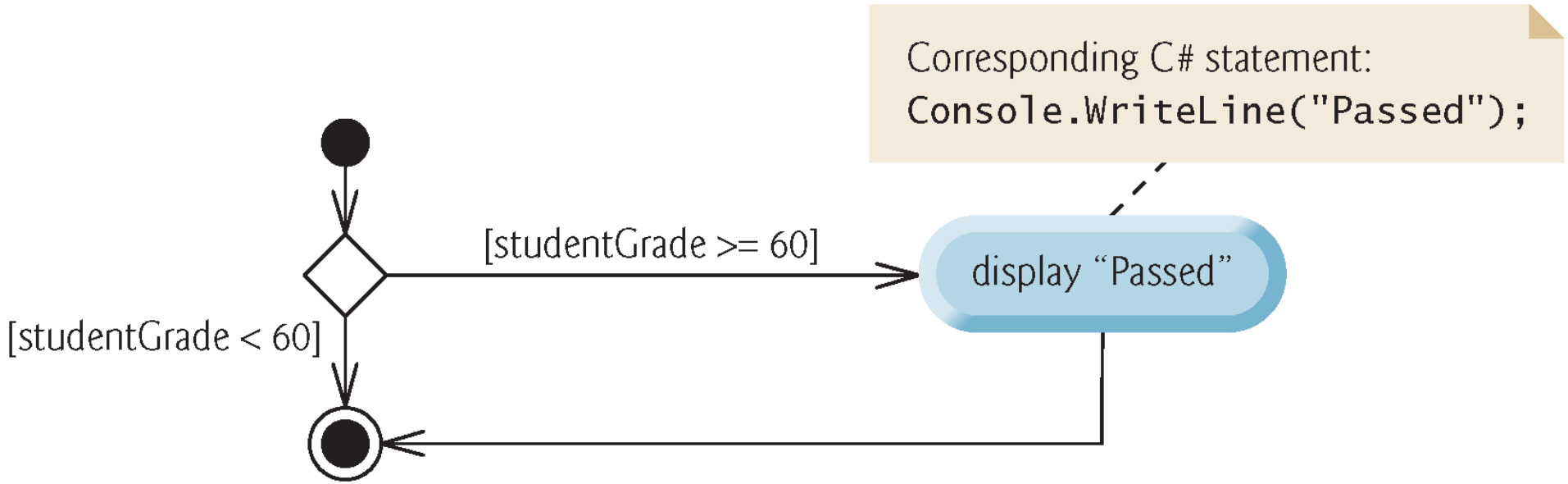
- C# provides four **iteration statements** (sometimes called **repetition statements** or **looping statements**) that enable programs to perform statements repeatedly as long as a condition (called the **loop-continuation condition**) remains *true*.
  - `while`, `do`…`while`, `for` and `foreach` statements.
- The `while`, `for` and `foreach` statements perform the action (or group of actions) in their bodies *zero or more times*.
- The `do`…`while` statement performs the action (or group of actions) in its body *one or more times*.

`if` Single-Selection Statement

- *if grade is greater than or equal to 60*
  display "Passed"
- The preceding pseudocode if statement may be written in C# as:
  - ```
    if (grade >= 60)
    {
       Console.WriteLine("Passed");
    }
    ```
- The C# code corresponds closely to the pseudocode.
    ```

`if` Single-Selection Statement (Cont..)

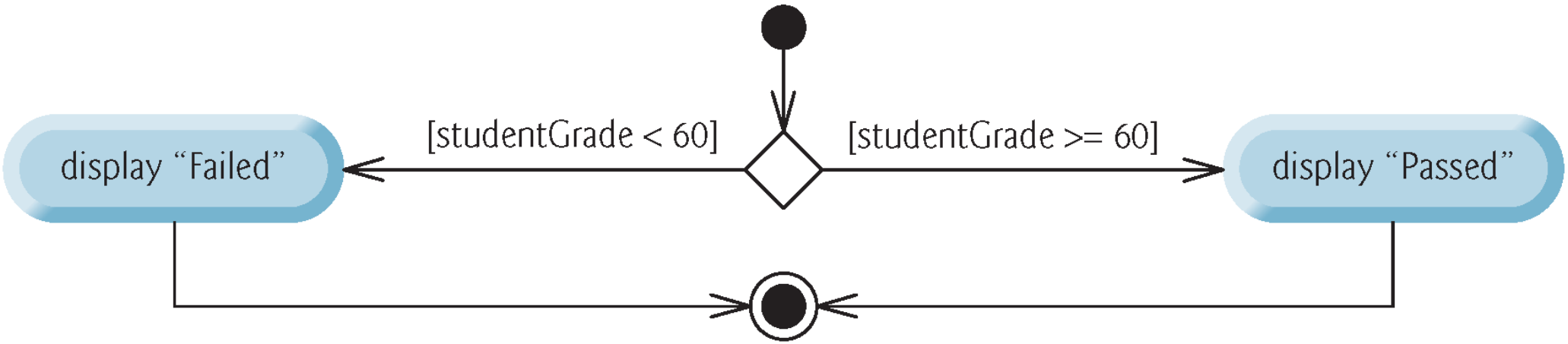
- ***bool Simple Type***
- A decision can be based on *any* expression that evaluates to **true** or **false**.
- C# provides the simple type **bool** for Boolean variables that can hold only the values `true` and `false`—each of these is a C# keyword.



`if...else` Double-Selection Statement

- *if grade is greater than or equal to 60*
  display "Passed"
  else
  display "Failed"
- The preceding `if...else` pseudocode statement can be written in C# as

```
if (grade >= 60)
{
   Console.WriteLine("Passed");
}
else
{
   Console.WriteLine("Failed");
}
```



Nested `if...else` Statements

- Nested `if...else` Statements
- An app can test multiple cases with nested `if...else` statements:
- *if grade is greater than or equal to 90*
  display "A"
  else
  if grade is greater than or equal to 80
  display "B"
  else
  if grade is greater than or equal to 70
  display "C"
  else
  if grade is greater than or equal to 60
  display "D"
  else
  display "F"

Nested if…else Statements

```
if (studentGrade >= 90)
{
   Console.WriteLine("A");
}
else if (studentGrade >= 80)
{
   Console.WriteLine("B");
}
else if (studentGrade >= 70)
{
   Console.WriteLine("C");
}
else if (studentGrade >= 60)
{
   Console.WriteLine("D");
}
else
{
   Console.WriteLine("F");
}
```

Blocks

- Statements contained within a pair of braces ({ and }) form a **block**:

```
if (studentGrade >= 60)
{
   Console.WriteLine("Passed");
}
else
{
   Console.WriteLine("Failed");
   Console.WriteLine("You must take this course again.");
}
```

- Without the braces, the last statement in the `else` block would execute regardless of whether the grade was less than 60.

Conditional Operator (?:)

- The conditional operator (`?:`) can be used in place of an `if…else` statement.
- ```
  Console.WriteLine(
      studentGrade >= 60 ? "Passed" : "Failed");
  ```
  - The first operand is a `boolean` expression that evaluates to `true` or `false`.
  - The second operand is the value if the expression is `true`
  - The third operand is the value if the expression is `false`.
  ```

`Student` Class: Nested `if`…`else` Statements

- The following example demonstrates a nested `if`…`else` statement that determines a student's letter grade based on the student's average in a course.
- Read-only property `LetterGrade` (lines 39–68) uses *nested `if`…`else` statements* to determine the `Student`'s letter grade based on the `Student`'s average.
- A read-only property provides only a `get` accessor.
- Local variable `letterGrade` is initialized to `string.Empty` (line 43), which represents the empty `string` (that is, a `string` containing no characters).

```
1  // Fig. 5.5: Student.cs
2  // Student class that stores a student name and average.
3  using System;
4
5  class Student
6  {
7     public string Name { get; set; } // property
8     private int average; // instance variable
9
10    // constructor initializes Name and Average properties
11    public Student(string studentName, int studentAverage)
12    {
13       Name = studentName;
14       Average = studentAverage; // sets average instance variable
15    }
16
```

```
17      // property to get and set instance variable average
18      public int Average
19      {
20         get // returns the Student's average
21         {
22            return average;
23         }
24         set  // sets the Student's average
25         {
26            // validate that value is > 0 and <= 100; otherwise,
27            // keep instance variable average's current value
28            if (value > 0)
29            {
30               if (value <= 100)
31               {
32                  average = value; // assign to instance variable
33               }
34            }
35         }
36      }
37
```

```
38      // returns the Student's letter grade, based on the average
39      string LetterGrade
40      {
41         get
42         {
43            string letterGrade = string.Empty; // string.Empty is ""
44
45            if (average >= 90)
46            {
47               letterGrade = "A";
48            }
49            else if (average >= 80)
50            {
51               letterGrade = "B";
52            }
```

```
53              else if (average >= 70)
54              {
55                 letterGrade = "C";
56              }
57              else if (average >= 60)
58              {
59                 letterGrade = "D";
60              }
61              else
62              {
63                 letterGrade = "F";
64              }
65
66              return letterGrade;
67          }
68      }
69  }
```

```
1  // Fig. 5.6: StudentTest.cs
2  // Create and test Student objects.
3  using System;
4
5  class StudentTest
6  {
7     static void Main()
8     {
9        Student student1 = new Student("Jane Green", 93);
10       Student student2 = new Student("John Blue", 72);
11
12       Console.Write($"{student1.Name}'s letter grade equivalent of ");
13       Console.WriteLine($"{student1.Average} is {student1.LetterGrade}");
14       Console.Write($"{student2.Name}'s letter grade equivalent of ");
15       Console.WriteLine($"{student2.Average} is {student2.LetterGrade}");
16    }
17 }
```

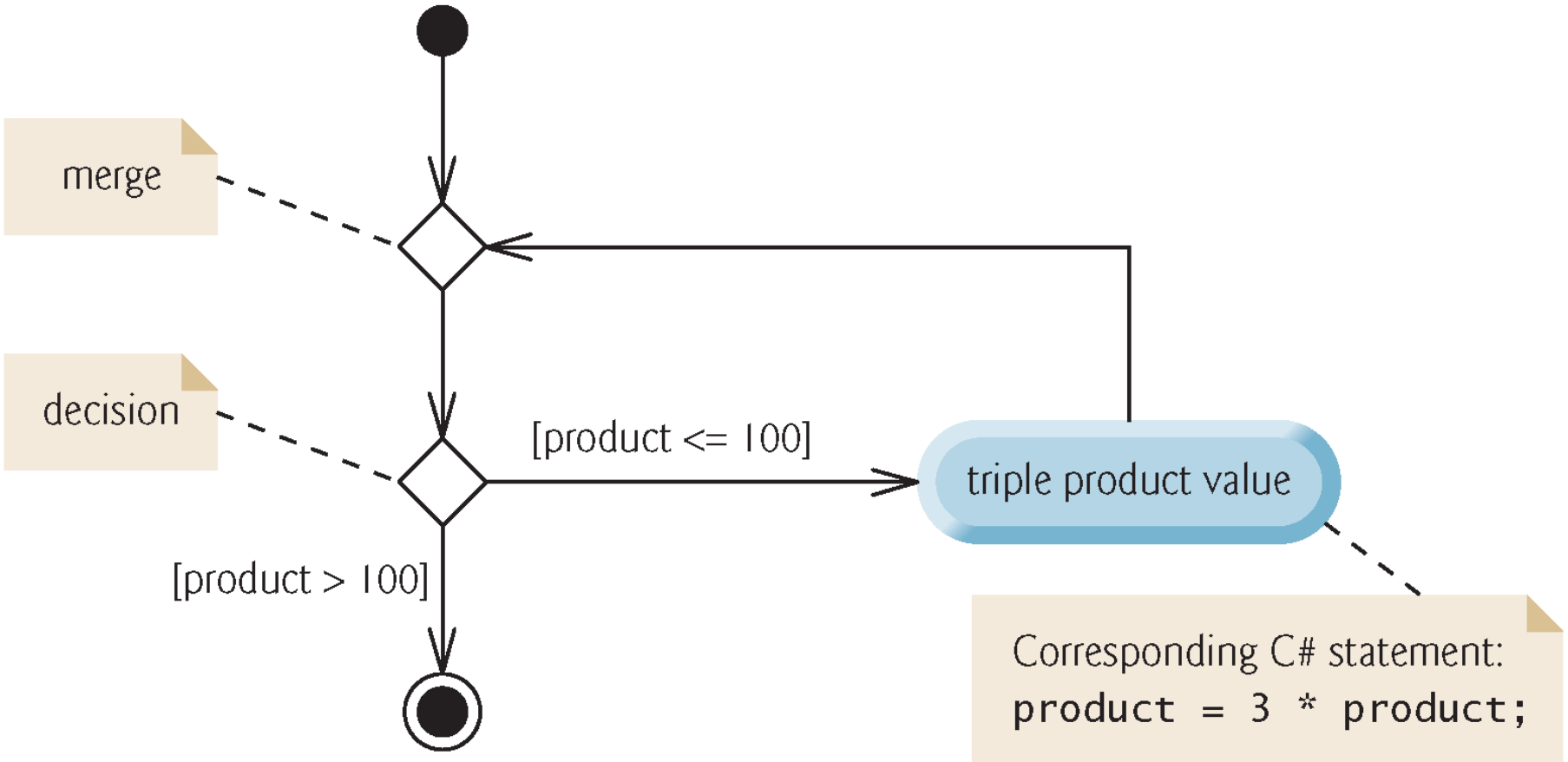
```
Jane Green's letter grade equivalent of 93 is A
John Blue's letter grade equivalent of 72 is C
```

`while` Iteration Statement

- A iteration statement allows you to specify that an app should repeat an action:
- *While there are more items on my shopping list*
  put next item in cart and cross it off my list
- As an example of C#'s `while` iteration statement, consider a code segment designed to find the first power of 3 larger than 100:
-

```
int product = 3;

while (product <= 100)
{
   product = 3 * product;
}
```



Formulating Algorithms: Counter-Controlled Iteration

- Consider the following problem statement:
- A class of 10 students took a quiz. The grades (integers in
- the range 0 to 100) for this quiz are available to you.
- Determine the class average on the quiz.
- The algorithm must input each grade, keep track of the total of all grades input, perform the averaging calculation and display the result.

Pseudocode Algorithm with Counter-Controlled Iteration

- Pseudocode Algorithm with Counter-Controlled Iteration
- We use counter-controlled iteration to input the grades one at a time.
- A variable called a counter controls the number of times a set of statements will execute.
- Counter-controlled iteration is often called definite iteration, because the number of iterations is known before the loop begins executing.
- A **total** is a variable used to accumulate the sum of several values.
- A **counter** is a variable used to count.

```
1   Set total to zero
2   Set grade counter to one
3
4   While grade counter is less than or equal to 10
5       Prompt the user to enter the next grade
6       Input the next grade
7       Add the grade into the total
8       Add one to the grade counter
9
10  Set the class average to the total divided by 10
11  Display the class average
```

```
1  // Fig. 5.9: ClassAverage.cs
2  // Solving the class-average problem using counter-controlled iteration.
3  using System;
4
5  class ClassAverage
6  {
7     static void Main()
8     {
9        // initialization phase
10       int total = 0; // initialize sum of grades entered by the user
11       int gradeCounter = 1; // initialize grade # to be entered next
12
```

```
13          // processing phase uses counter-controlled iteration
14          while (gradeCounter <= 10) // loop 10 times
15          {
16             Console.Write("Enter grade: "); // prompt
17             int grade = int.Parse(Console.ReadLine()); // input grade
18             total = total + grade; // add the grade to total
19             gradeCounter = gradeCounter + 1; // increment the counter by 1
20          }
21
22          // termination phase
23          int average = total / 10; // integer division yields integer result
24
25          // display total and average of grades
26          Console.WriteLine($"\nTotal of all 10 grades is {total}");
27          Console.WriteLine($"Class average is {average}");
28       }
29    }
```

```
Enter grade: 88
Enter grade: 79
Enter grade: 95
Enter grade: 100
Enter grade: 48
Enter grade: 88
Enter grade: 92
Enter grade: 83
Enter grade: 90
Enter grade: 85

Total of all 10 grades is 848
Class average is 84
```

Integer Division and Truncation

- Dividing two integers results in integer division.
- Any fractional part of the result is lost (i.e., truncated, not rounded).

Formulating Algorithms: Sentinel-Controlled Iteration

- Consider the following problem:
  - Develop a class-averaging app that processes grades for an arbitrary number of students each time it's run.
- In this example, no indication is given of how many grades the user will enter during the app's execution.
- A sentinel value can be entered to indicate "end of data entry." This is called sentinel-controlled iteration.
- Sentinel-controlled iteration is often called indefinite iteration because the number of iterations is not known before the loop begins.

```
1   Initialize total to zero
2   Initialize counter to zero
3
4   Prompt the user to enter the first grade
5   Input the first grade (possibly the sentinel)
6
7   While the user has not yet entered the sentinel
8       Add this grade into the running total
9       Add one to the grade counter
10      Prompt the user to enter the next grade
11      Input the next grade (possibly the sentinel)
12
13  If the counter is not equal to zero
14      Set the average to the total divided by the counter
15      Print the average
16  Else
17      Print "No grades were entered"
```

```
1  // Fig. 5.11: ClassAverage.cs
2  // Solving the class-average problem using sentinel-controlled iteration.
3  using System;
4
5  class ClassAverage
6  {
7     static void Main()
8     {
9        // initialization phase
10       int total = 0; // initialize sum of grades
11       int gradeCounter = 0; // initialize # of grades entered so far
12
13       // processing phase
14       // prompt for input and read grade from user
15       Console.Write("Enter grade or -1 to quit: ");
16       int grade = int.Parse(Console.ReadLine());
17
```

```
18          // loop until sentinel value is read from the user
19          while (grade != -1)
20          {
21             total = total + grade; // add grade to total
22             gradeCounter = gradeCounter + 1; // increment counter
23
24             // prompt for input and read grade from user
25             Console.Write("Enter grade or -1 to quit: ");
26             grade = int.Parse(Console.ReadLine());
27          }
28
```

```
29          // termination phase
30          // if the user entered at least one grade...
31          if (gradeCounter != 0)
32          {
33             // use number with decimal point to calculate average of grades
34             double average = (double) total / gradeCounter;
35
36             // display the total and average (with two digits of precision)
37             Console.WriteLine(
38                $"\nTotal of the {gradeCounter} grades entered is {total}");
39             Console.WriteLine($"Class average is {average:F}");
40          }
41          else // no grades were entered, so output error message
42          {
43             Console.WriteLine("No grades were entered");
44          }
45       }
46    }
```

```
Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1

Total of the 3 grades entered is 257
Class average is 85.67
```

Converting Between Simple Types Explicitly and Implicitly

- To perform a floating-point calculation with integer values, we temporarily treat these values as floating-point numbers.
- A unary cast operator such as `(double)` performs explicit conversion.
- C# performs an operation called promotion (or implicit conversion) on selected operands for use in the expression.
- The cast operator is formed by placing parentheses around the name of a type.
- This operator is a unary operator (i.e., an operator that takes only one operand).
- Cast operators have the second highest precedence.

Formulating Algorithms: Nested Control Statements

- We've seen that control statements can be *stacked* on top of one another (in sequence).
- In this case study, we examine the only other structured way control statements can be connected, namely, by **nesting** one control statement within another.

Formulating Algorithms: Nested Control Statements

- A college offers a course that prepares students for the state licensing exam for real estate brokers. Last year, 10 of the students who completed this course took the exam. The college wants to know how well its students did on the exam. You've been asked to write an app to summarize the results. You've been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam or a 2 if the student failed.

```
1   Initialize passes to zero
2   Initialize failures to zero
3   Initialize student counter to one
4
5   While student counter is less than or equal to 10
6       Prompt the user to enter the next exam result
7       Input the next exam result
8
9       If the student passed
10          Add one to passes
11      Else
12          Add one to failures
13
14      Add one to student counter
15
16  Display the number of passes
17  Display the number of failures
18
19  If more than eight students passed
20      Display "Bonus to instructor!"
```

```
1  // Fig. 5.13: Analysis.cs
2  // Analysis of examination results, using nested control statements.
3  using System;
4
5  class Analysis
6  {
7     static void Main()
8     {
9        // initialize variables in declarations
10       int passes = 0; // number of passes
11       int failures = 0; // number of failures
12       int studentCounter = 1; // student counter
13
```

```
14          // process 10 students using counter-controlled iteration
15          while (studentCounter <= 10)
16          {
17             // prompt user for input and obtain a value from the user
18             Console.Write("Enter result (1 = pass, 2 = fail): ");
19             int result = int.Parse(Console.ReadLine());
20
21             // if...else is nested in the while statement
22             if (result == 1)
23             {
24                passes = passes + 1; // increment passes
25             }
26             else
27             {
28                failures = failures + 1; // increment failures
29             }
30
31             // increment studentCounter so loop eventually terminates
32             studentCounter = studentCounter + 1;
33          }
```

```
34
35          // termination phase; prepare and display results
36          Console.WriteLine($"Passed: {passes}\nFailed: {failures}");
37
38          // determine whether more than 8 students passed
39          if (passes > 8)
40          {
41             Console.WriteLine("Bonus to instructor!");
42          }
43       }
44    }
```

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 9
Failed: 1
Bonus to instructor!
```

```
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Passed: 5
Failed: 5
```

Compound Assignment Operators

- C# provides several **compound assignment operators**
- For example, you can abbreviate the statement

```
c = c + 3;
```

- with the **addition compound assignment operator**, **+=**, as

```
c += 3;
```

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Operator	Sample expression	Explanation
++ (prefix increment)	`++a`	Increments `a` by 1 and uses the new value of `a` in the expression in which `a` resides.
++ (postfix increment)	`a++`	Increments `a` by 1, but uses the original value of `a` in the expression in which `a` resides.
-- (prefix decrement)	`--b`	Decrements `b` by 1 and uses the new value of `b` in the expression in which `b` resides.
-- (postfix decrement)	`b--`	Decrements `b` by 1 but uses the original value of `b` in the expression in which `b` resides.

```
1  // Fig. 5.16: Increment.cs
2  // Prefix increment and postfix increment operators.
3  using System;
4
5  class Increment
6  {
7     static void Main()
8     {
9        // demonstrate postfix increment operator
10       int c = 5; // assign 5 to c
11       Console.WriteLine($"c before postincrement: {c}"); // displays 5
12       Console.WriteLine($"    postincrementing c: {c++}"); // displays 5
13       Console.WriteLine($" c after postincrement: {c}"); // displays 6
14
15       Console.WriteLine(); // skip a line
16
```

```
17          // demonstrate prefix increment operator
18          c = 5; // assign 5 to c
19          Console.WriteLine($" c before preincrement: {c}"); // displays 5
20          Console.WriteLine($"     preincrementing c: {++c}"); // displays 6
21          Console.WriteLine($"  c after preincrement: {c}"); // displays 6
22       }
23    }
```

```
c before postincrement: 5
    postincrementing c: 5
 c after postincrement: 6

 c before preincrement: 5
     preincrementing c: 6
  c after preincrement: 6
```

Operators						Associativity	Type
`.`	`new`	`++`*(postfix)*	`--`*(postfix)*			left to right	highest precedence
`++`	`--`	`+`	`-`	(*type*)		right to left	unary prefix
`*`	`/`	`%`				left to right	multiplicative
`+`	`-`					left to right	additive
`<`	`<=`	`>`	`>=`			left to right	relational
`==`	`!=`					left to right	equality
`?:`						right to left	conditional
`=`	`+=`	`-=`	`*=`	`/=`	`%=`	right to left	assignment

Simple Types

- C# requires *all* variables to have a type.
- Instance variables of types `char`, `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, and decimal are all given the value `0` by default.
- Instance variables of type `bool` are given the value `false` by default.
- Reference-type instance variables are initialized by default to the value `null`.